

PRACTICAL- 3 (A)

AIM

Guided Search Techniques:

- a. Write a python code for implementing 8 Puzzle Problem using Best First Search.

THEORY

The 8 Puzzle Problem is a classical Artificial Intelligence problem used to demonstrate search techniques. It consists of a 3×3 grid containing eight numbered tiles and one blank space. The objective is to rearrange the tiles from an initial configuration to a goal configuration by sliding tiles into the blank space.

Best First Search is a guided search technique that selects the next node to explore based on a heuristic function. The heuristic estimates how close the current state is to the goal state.

In the 8 puzzle problem, a common heuristic used is the number of misplaced tiles. The algorithm always expands the state that appears closest to the goal according to this heuristic value.

Best First Search uses a priority queue to store states and selects the state with the smallest heuristic value first. This helps the algorithm move toward the goal faster compared to uninformed search methods.

SOFTWARE USED

Jupyter Notebook

CODE

```
import heapq
```

```
goal_state = [[1,2,3],  
              [4,5,6],  
              [7,8,0]]
```

```
def heuristic(state):  
    misplaced = 0  
    for i in range(3):
```

```
    for j in range(3):
        if state[i][j] != 0 and state[i][j] != goal_state[i][j]:
            misplaced += 1
    return misplaced
```

```
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j
```

```
def best_first_search(start):
    visited = []
    pq = []

    heapq.heappush(pq,(heuristic(start), start))
```

```
    while pq:
        h, current = heapq.heappop(pq)
```

```
        if current == goal_state:
            print("Goal State Reached")
            for row in current:
                print(row)
            return
```

```
        visited.append(current)
```

```
        x, y = find_blank(current)
```

```

moves = [(0,1),(0,-1),(1,0),(-1,0)]

for dx, dy in moves:
    nx, ny = x + dx, y + dy

    if 0 <= nx < 3 and 0 <= ny < 3:
        new_state = [row[:] for row in current]
        new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]

        if new_state not in visited:
            heapq.heappush(pq,(heuristic(new_state), new_state))

start_state = [[1,2,3],
                [4,0,6],
                [7,5,8]]

best_first_search(start_state)

```

OUTPUT

Goal State Reached

[1, 2, 3]

[4, 5, 6]

[7, 8, 0]

PRACTICAL- 3 (B)

AIM

Guided Search Techniques:

b. Write a python code for implementing 8 Puzzle Problem using A* Search algorithm.

THEORY

The A* (A-star) search algorithm is a widely used informed search algorithm in Artificial Intelligence. It combines the advantages of uniform cost search and heuristic search to find the shortest path to the goal state.

The evaluation function used in A* is:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = cost from start node to current node
- $h(n)$ = heuristic estimate from current node to goal
- $f(n)$ = total estimated cost of the solution path

For the 8 puzzle problem, the heuristic commonly used is the number of misplaced tiles or Manhattan distance.

The algorithm uses a priority queue to select the state with the smallest $f(n)$ value. This allows A* to efficiently explore the search space and guarantees finding the optimal solution when the heuristic is admissible.

SOFTWARE USED

Jupyter Notebook

CODE

```
import heapq
```

```
goal = [[1,2,3],  
        [4,5,6],  
        [7,8,0]]
```

```
def heuristic(state):  
    count = 0  
    for i in range(3):  
        for j in range(3):  
            if state[i][j] != 0 and state[i][j] != goal[i][j]:  
                count += 1  
    return count
```

```
def find_blank(state):  
    for i in range(3):  
        for j in range(3):  
            if state[i][j] == 0:  
                return i, j
```

```
def a_star(start):
```

```
    visited = []
```

```
    pq = []
```

```
    heapq.heappush(pq,(0,start,0))
```

```
    while pq:
```

```
        f,state,g = heapq.heappop(pq)
```

```
        if state == goal:
```

```
            print("Goal Reached")
```

```
            for row in state:
```

```
                print(row)
```

```
            return
```

```
visited.append(state)
```

```
x,y = find_blank(state)
```

```
moves = [(0,1),(0,-1),(1,0),(-1,0)]
```

```
for dx,dy in moves:
```

```
    nx,ny = x+dx,y+dy
```

```
    if 0<=nx<3 and 0<=ny<3:
```

```
        new_state = [row[:] for row in state]
```

```
        new_state[x][y],new_state[nx][ny] = new_state[nx][ny],new_state[x][y]
```

```
        if new_state not in visited:
```

```
            g_new = g + 1
```

```
            f_new = g_new + heuristic(new_state)
```

```
            heapq.heappush(pq,(f_new,new_state,g_new))
```

```
start = [[1,2,3],
```

```
         [4,0,6],
```

```
         [7,5,8]]
```

```
a_star(start)
```

OUTPUT

Goal State Reached

[1, 2, 3]

[4, 5, 6]

[7, 8, 0]